ОРГАНІЗАЦІЯ КОМП'ЮТЕРНИХ МЕРЕЖ

Лабораторна робота №1

Розробка мережевого клієнт-серверного додатку з використанням сокетного інтерфейсу (Socket API) для протоколів TCP та UDP

Варіант 16 - Weather Mock

Виконав: Терніков Дмитро Романович
Група: П-423

18 жовтня 2026 року

1. Мета, середовище та архітектура

Мета: освоїти BSD Socket API, створити власний прикладний протокол поверх TCP і UDP, реалізувати паралельне обслуговування клієнтів та дослідити таймаути і втрати датаграм.

Роботу виконано на комп'ютері користувача, Windows 10 x64, Python 3 зі стандартною бібліотекою. Захоплення: Wireshark 4.6.9, Npcap 1.88, dumpcap на адаптері NPF_Loopback. Адреса досліду 127.0.0.1; TCP-порт 9000, UDP-порт 9001. Зовнішній сервіс погоди не використовується.

TCPServer використовує ThreadingMixIn: кожний прийнятий клієнт обслуговується окремим потоком. UDP-приймач працює паралельно з TCP, а окремий потік кожні 2 секунди розсилає штормові попередження підписникам. Журнал містить час, IP, ефемерний порт, команди, завершення сеансів та відкинуті запити.

Компонент	Призначення
weather_mock.py server	TCP + UDP, логування, mock-погода, Emergency Alert
weather_mock.py tcp	Інтерактивні GET_WEATHER City та EXIT
weather_mock.py udp	100 Packet #N, ACK, таймаут 1 с, підрахунок втрат
weather_mock.py alerts	Підписка й прийом попереджень
test_weather.py	5 інтеграційних тестів через справжні сокети
run_experiment.py	Повторення досліду та перегляд збережених результатів

Погода генерується детерміновано через SHA-256 назви міста: температура від -20 до 40 °C, вологість 20-100%, вітер 0-30 м/с. Прапорець mock=true однозначно вказує на навчальні дані.

Обмеження: це навчальний сервер без автентифікації, шифрування та обмеження кількості потоків. Для досліду він слухає лише loopback. Результати локальної мережі не є оцінкою поведінки інтернет-з'єднання.

2. Прикладний протокол і запуск

TCP: кодування UTF-8; запит завершується LF, відповідь - JSON + LF. Одне з'єднання підтримує кілька команд. Межі TCP-сегментів не визначають межі команд: readline збирає рядок із потоку. Максимальний запит - 4096 байтів разом із LF; надмірний або незавершений кадр спричиняє помилку та закриття.

```
GET_WEATHER Kyiv
{"status":"OK","city":"Kyiv","temperature_c":2,
 "humidity_pct":58,"wind_m_s":15.4,"mock":true}
EXIT
{"status":"BYE"}
```

Невідома команда: UNKNOWN_COMMAND; некоректне місто: INVALID_CITY; невірне кодування: INVALID_UTF8. Міста з пробілами та кирилицею підтримуються. Після BYE сервер закриває сеанс; клієнт закриває файловий об'єкт і сокет.

UDP: Packet #N отримує JSON {type: ACK, seq: N}. За drop_every=10 сервер не відповідає на номери, кратні 10. Відкидання детерміноване за номером, тому дослід відтворюваний. Клієнт послідовно надсилає 100 запитів; перевіряє джерело та номер ACK, ігнорує сторонні або застарілі відповіді до абсолютного дедлайну 1 с.

SUBSCRIBE реєструє адресу UDP-слухача та повертає SUBSCRIBED. Сервер надсилає EMERGENCY_ALERT з номером і текстом MOCK: storm warning кожному слухачу окремою UDP-датаграмою. Це unicast-розсилання за списком підписок. UNSUBSCRIBE скасовує підписку; без поновлення вона спливає за 300 с.

У різних терміналах із папки проєкту (Python 3):

```
python weather_mock.py server
python weather_mock.py tcp
python weather_mock.py alerts
python weather_mock.py udp --output result.json
python -m unittest -v test_weather
```

На цьому комп'ютері launch.ps1 знаходить наявний Python із середовища Codex; наприклад ./launch.ps1 server або ./launch.ps1 tcp. Контрольний запуск: server --drop-every 0. Зупинка сервера: Ctrl+C. Точні команди та структура доказів наведені також у README.md.

3. Реальні результати TCP

Одночасно відкрито 4 TCP-сеанси. Бар’єр у досліді змушує всі чотири з’єднання бути відкритими до надсилання запитів та завершення перевірки. Ефемерні порти: 64341, 64343, 64344, 64342. У журналі видно окремі потоки та перекриття часу сеансів.

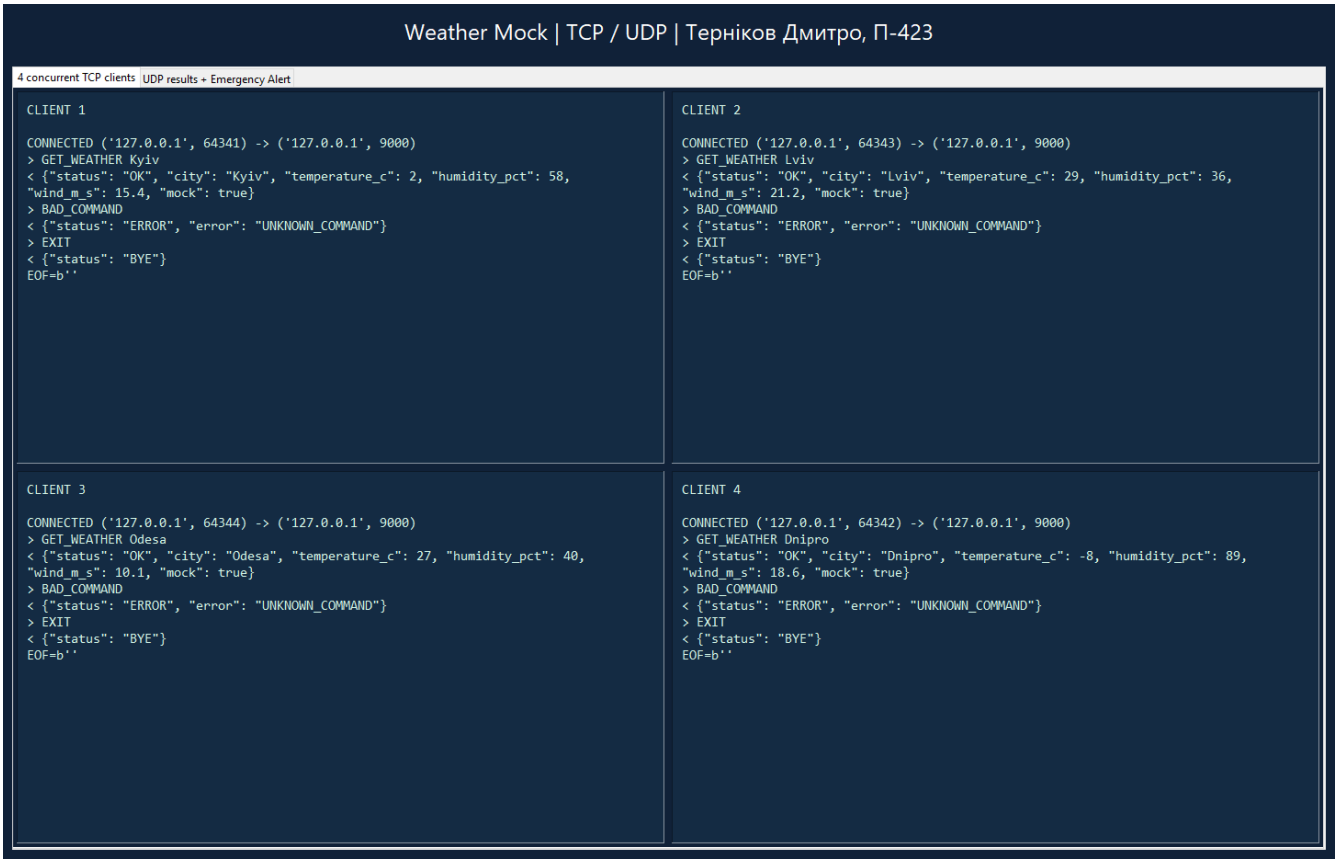


Рис. 1. Скриншот панелі з фактичними записами чотирьох паралельних TCP-сеансів. Панель відображає збережені результати, а не чотири відкриті термінали.

Для Kyiv отримано 2 °C, 58%, 15,4 м/с; для Lviv 29 °C, 36%, 21,2 м/с; для Odesa 27 °C, 40%, 10,1 м/с; для Dnipro -8 °C, 89%, 18,6 м/с. Невідома команда повернула помилку. Після EXIT кожний клієнт отримав BYE та EOF.

Окремо запущено саме консольний TCP-клієнт зі stdin: GET_WEATHER Київ, EXIT. Його справжній вивід збережено у evidence/tcp_client.txt. Тести також перевірили фрагментацію запиту і кілька команд в одному надсиланні.

4. UDP: таймаути, втрати та попередження

Режим	Надіслано	Отримано	Втрачено	Частка
drop_every=10	100	90	10	10%
drop_every=0	100	100	0	0%

Packet Loss Ratio = (100 - 90) / 100 × 100% = 10%. Таймаути: #10, #20, #30, #40, #50, #60, #70, #80, #90, #100. Виміряний час очікування: від 1000 до 1016 мс; перевищення 1000 мс пояснюється плануванням ОС. Середній прикладний RTT успішних відповідей: 0.167 мс.

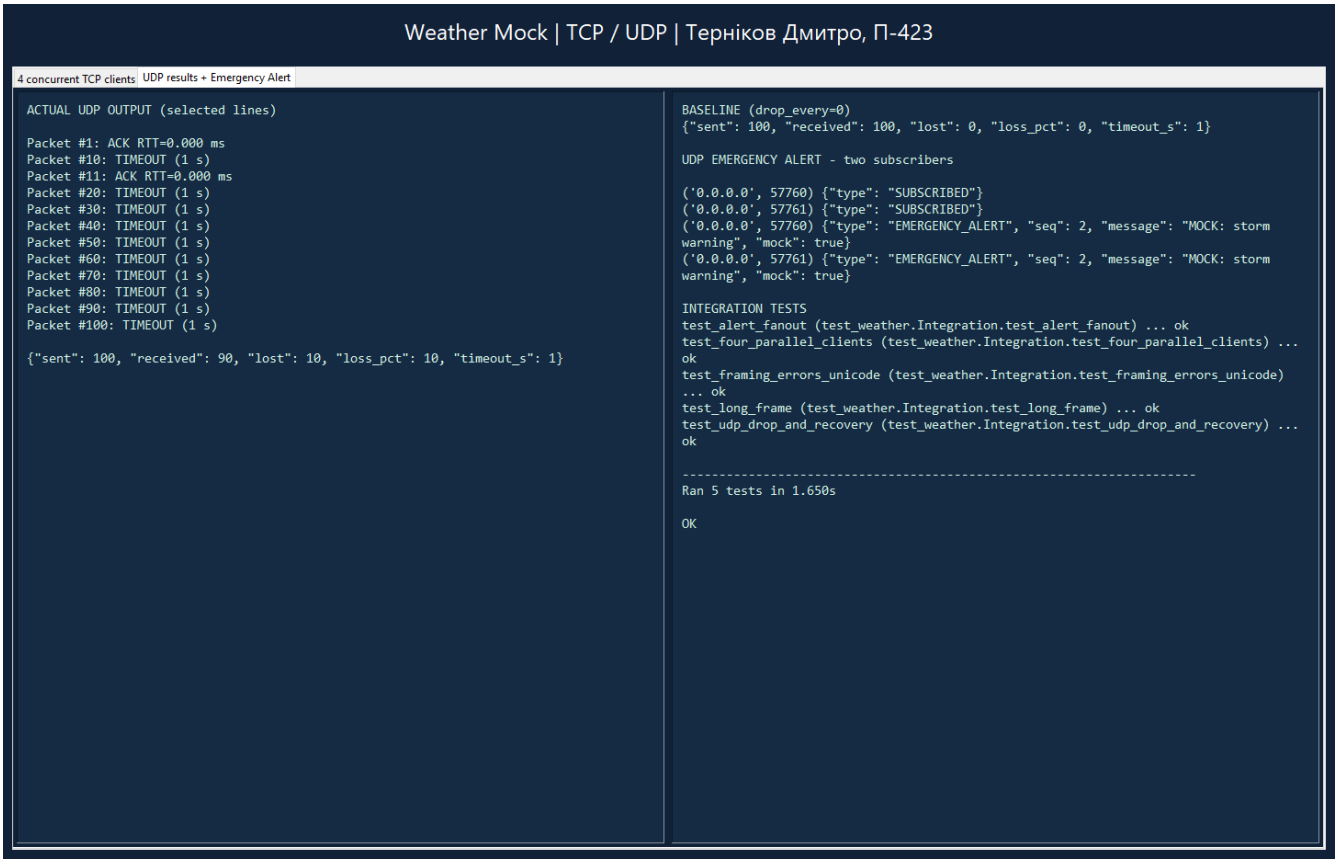


Рис. 2. Знімок вікна зі справжніми таймаутами, контрольним запуском, двома слухачами Emergency Alert і підсумком тестів.

Два підписники (порти 57760 та 57761) отримали штормове попередження seq=2. Дані збережено у alerts.txt та кадрах 96, 98. UDP не має прапорів SYN/ACK/FIN; його заголовок містить порти, довжину й контрольну суму.

Вимірюється відсутність прикладної відповіді, а не фізична втрата датаграми в мережі: відкинутий сервером запит залишається видимим у дампі. Контрольний дослід підтвердив 0% втрат при вимкненому моделюванні.

5. Аналіз трафіку Wireshark

Захоплення виконано dumpcap із пакета Wireshark, фільтр tcp port 9000 or udp port 9001; файл traffic.pcapng містить 489 кадрів. Відкрито у Wireshark. Для першого потоку застосовано tcp.stream == 0.

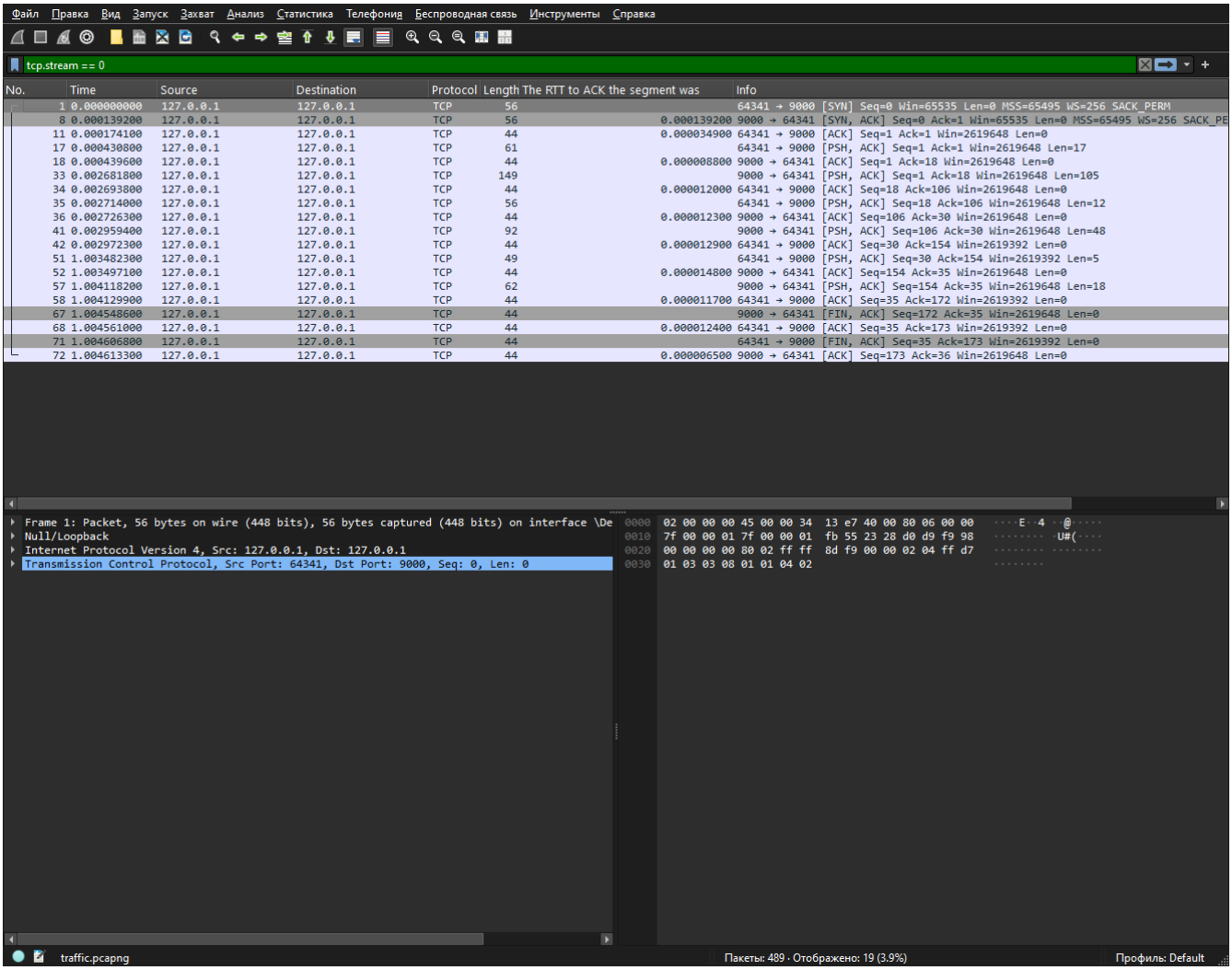


Рис. 3. Справжнє вікно Wireshark: SYN (кадр 1), SYN+ACK (8), ACK (11), прикладні дані та FIN+ACK під час закриття.

Кадр	Напрямок	Прапори	Значення
1	64341 → 9000	0x0002 SYN	Початок з’єднання
8	9000 → 64341	0x0012 SYN, ACK	Відповідь сервера
11	64341 → 9000	0x0010 ACK	Завершення handshake
67 / 71	Сервер / клієнт	0x0011 FIN, ACK	Коректне завершення

5.1. Датаграма штормового попередження

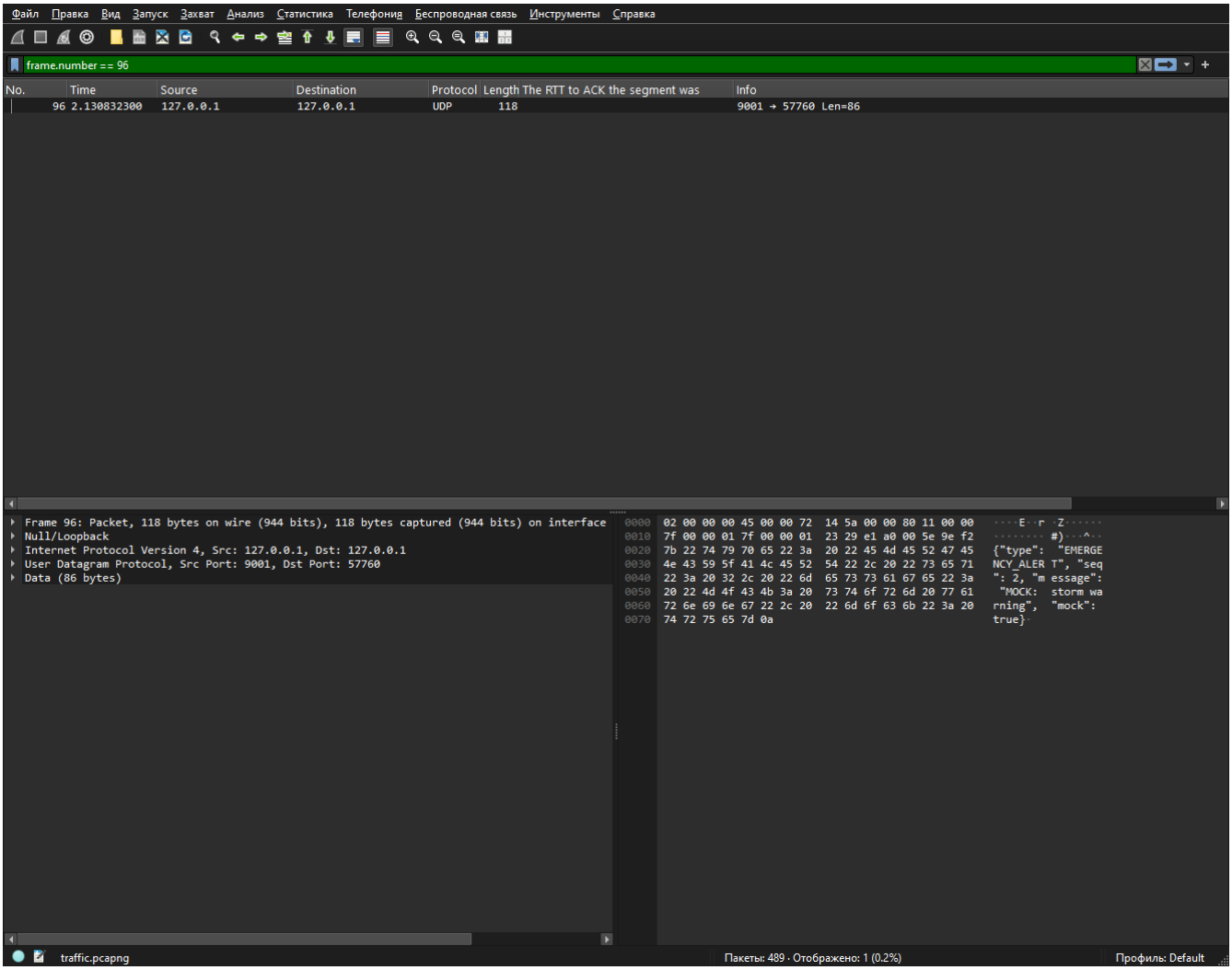


Рис. 4. Справжня UDP-датаграма Emergency Alert, фільтр frame.number == 96.

Кадр 96: 127.0.0.1:9001 → 127.0.0.1:57760, UDP Length=94 байти: 8 байтів заголовка та 86 байтів JSON із LF. У кадрі 98 те саме попередження надіслано другому підписнику. З'єднання та тристороннього рукостискання UDP не встановлює.

```
{"type": "EMERGENCY_ALERT", "seq": 2,
 "message": "MOCK: storm warning", "mock": true}
```

Докази: traffic.pcapng - первинне захоплення; packet_fields.csv - експорт полів; tcp_packets.txt - список першого потоку; udp_packet.txt - повний розбір кадру 96. Packet #10 присутній у мережевому захопленні, але сервер не надсилає відповідний ACK. Наступний запит починається після таймауту.

6. Тестування та висновки

```
test_alert_fanout (test_weather.Integration.test_alert_fanout) ... ok
test_four_parallel_clients (test_weather.Integration.test_four_parallel_clients) ... ok
test_framing_errors_unicode (test_weather.Integration.test_framing_errors_unicode) ... ok
test_long_frame (test_weather.Integration.test_long_frame) ... ok
test_udp_drop_and_recovery (test_weather.Integration.test_udp_drop_and_recovery) ... ok

----------------------------------------------------------------------
Ran 5 tests in 1.650s

OK
```

Перевірено: одночасні сеанси й EXIT/EOF; правильне складання фрагментованого TCP-запиту та розбір кількох рядків; кирилицю й помилки протоколу; обмеження розміру кадру; UDP-таймаут і відновлення на наступному номері; розсилання двом підписникам. Усі 5 інтеграційних тестів пройшли на реальних loopback-сокетах.

Результат: створено багатопотоковий TCP-сервер Weather Mock, консольний TCP-клієнт, UDP-приймач, клієнт дослідження втрат та модуль штормових попереджень. Фреймінг через LF коректно відокремлює повідомлення в TCP-потоці. Локальні TCP-сеанси завершуються BYE і FIN/ACK; UDP-клієнт самостійно виявляє відсутність відповіді за таймаутом.

За відкидання кожного десятого UDP-запиту зафіксовано 10% втрат відповідей, а без відкидання - 0%. Збережено справжній дамп трафіку, журнали й скриншоти. Це підтверджує реалізацію вимог у локальному середовищі; багатокомп’ютерну мережу, мережеві збої та навантаження великої кількості клієнтів окремо не досліджували.

Git-репозиторій опубліковано на GitHub: https://github.com/Dmitry424/weather-mock-lab1, гілка main. У ньому доступні код, тести, звіт та докази досліду. Історичний крок публікації початкового коду перед розробкою не відтворювався заднім числом.

7. Відповіді на контрольні запитання

1. Чим accept() відрізняється від listen()? Що повертає accept()?

listen(backlog) переводить прив’язаний TCP-сокет у режим очікування вхідних з’єднань і налаштовує чергу. accept() забирає встановлене з’єднання із черги та повертає новий сокет для конкретного клієнта й адресу цього клієнта. Слухаючий сокет залишається доступним для наступних підключень.

2. Чому UDP не потребує listen() та accept()?

UDP не встановлює транспортного з’єднання. Після socket(SOCK_DGRAM) і bind() сервер отримує окремі датаграми через recvfrom(), який повертає також адресу відправника. Відповідь надсилається sendto(). Новий сокет для кожного клієнта не потрібний.

3. Socket Exhaustion і TIME_WAIT.

Socket Exhaustion - вичерпання ресурсів сокетів: дескрипторів, пам’яті або доступних ефемерних портів. Активна сторона закриття TCP зазвичай перебуває у TIME_WAIT протягом 2MSL, щоб старі сегменти не потрапили в новий сеанс і можна було повторно підтвердити FIN. Велика кількість коротких з’єднань може вичерпати доступні локальні порти для заданих кінцевих точок. Допомагають повторне використання встановлених з’єднань і своєчасне закриття ресурсів; SO_REUSEADDR не є універсальним усуненням TIME_WAIT.

4. Що станеться при раптовій зупинці процесу сервера?

Результат залежить від стану сокета та ОС. При скиданні з’єднання ядро надсилає RST, а клієнт може отримати ConnectionResetError. Проте саме завершення процесу, навіть kill -9, не гарантує RST: ядро закриває дескриптори і може виконати звичайне закриття з FIN, тоді recv() поверне b''. Непрочитані дані або абортивне закриття можуть спричинити RST. Якщо зник весь вузол чи зв’язок, негайного прапора може не бути, і збій виявиться через повторні передачі та таймаути. У цьому досліді зафіксовано штатне EXIT/FIN; аварійне завершення окремо не вимірювали.

7. Відповіді (продовження)

5. Неблокуючий recv() і таймаут.

У Python sock.setblocking(False) робить операції неблокуючими: за відсутності даних recv() піднімає BlockingIOError. Готовність зручно очікувати через selectors або select. sock.settimeout(1.0) задає обмежене очікування: за відсутності даних буде socket.timeout. У UDP-клієнті використано дедлайн time.monotonic(), щоб сторонні або запізнілі відповіді не подовжували загальний час очікування потрібного ACK.

6. Заголовки в TCP-потоці та framing problem.

TCP передає впорядкований потік байтів, а не повідомлення. Один sendall() може читатися кількома recv(), а кілька надсилань можуть об’єднатися в одному читанні. Прикладний заголовок, наприклад поле довжини, теж є частиною цього потоку, тому його потрібно накопичити й розібрати, а потім дочитати визначену кількість байтів. Фреймінг - правило встановлення меж повідомлень: фіксована довжина, префікс довжини або роздільник. У цій роботі використано LF і обмежений readline(); це не залежить від меж TCP-пакетів. UDP зберігає межі датаграм, але не гарантує доставку та порядок.

8. Матеріали та відтворення

Джерело завдання: наданий PDF «Лабораторна робота №1 (1)-1.pdf», копія assignment.pdf у проєкті. Первинні результати отримано запуском run_experiment.py на комп’ютері користувача 18.10.2026. Окремі тести запускаються python -m unittest -v test_weather.

У додатках наведено повний код сервера/клієнтів і тестів. Файл run_experiment.py та сценарій побудови звіту доступні у комплекті проєкту. Результати у звіті взяті зі збережених файлів evidence; значення не підставлялися як очікувані результати.

Для перегляду захоплення відкрийте evidence/traffic.pcapng у Wireshark. Для запуску серверів і клієнтів скористайтеся README.md та launch.ps1. На інших комп’ютерах достатньо встановленого Python 3 для основної програми.

Додаток А. Повний код сервера та клієнтів

weather_mock.py

```
"""Lab 1, variant 16. UTF-8 JSON replies; TCP commands end with LF."""
import argparse
import hashlib
import json
import logging
import re
import socket
import socketserver
import threading
import time

MAX_LINE = 4096


def encode(value):
    return (json.dumps(value, ensure_ascii=False) + '\n').encode('utf-8')


def weather(city):
    """Stable mock values, deliberately not a real weather forecast."""
    n = int.from_bytes(hashlib.sha256(city.casefold().encode()).digest()[:4], 'big')
    return dict(status='OK', city=city, temperature_c=n % 61 - 20,
                humidity_pct=20 + n // 61 % 81, wind_m_s=round(n // 4941 % 301 / 10, 1), mock=True)


def command(text):
    if text == 'EXIT':
        return dict(status='BYE')
    if text.startswith('GET_WEATHER '):
        city = text[12:].strip()
        if city and len(city) <= 100 and all(c.isalpha() or c in " -'." for c in city):
            return weather(city)
        return dict(status='ERROR', error='INVALID_CITY')
    return dict(status='ERROR', error='UNKNOWN_COMMAND')


class TCPHandler(socketserver.StreamRequestHandler):
    def handle(self):
        logging.info('CONNECT ip=%s port=%s', *self.client_address)
        self.request.settimeout(30)
        try:
            while True:
                # TCP is a stream: readline buffers split/combined TCP segments.
                raw = self.rfile.readline(MAX_LINE + 1)
                if not raw:
                    break
                if len(raw) > MAX_LINE or not raw.endswith(b'\n'):
                    self.wfile.write(encode(dict(status='ERROR',
error='FRAME_TOO_LONG_OR_INCOMPLETE')))
                    break
                try:
                    text = raw.decode('utf-8').strip()
                    answer = command(text)
                except UnicodeDecodeError:
                    text = '<invalid UTF-8>'
                    answer = dict(status='ERROR', error='INVALID_UTF8')
                logging.info('COMMAND client=%s command=%r', self.client_address, text)
                self.wfile.write(encode(answer))
                if answer['status'] == 'BYE':
                    break
        except (OSError, TimeoutError) as exc:
            logging.info('CLIENT_END %s %s', self.client_address, exc)
        finally:
            logging.info('DISCONNECT ip=%s port=%s', *self.client_address)


class TCPServer(socketserver.ThreadingMixIn, socketserver.TCPServer):
```

```
    allow_reuse_address = True
    daemon_threads = True


class UDPHandler(socketserver.BaseRequestHandler):
    def handle(self):
        data, sock = self.request
        try:
            text = data.decode('utf-8').strip()
        except UnicodeDecodeError:
            return
        if text == 'SUBSCRIBE':
            with self.server.lock:
                self.server.listeners[self.client_address] = time.monotonic()
            sock.sendto(encode(dict(type='SUBSCRIBED')), self.client_address)
        elif text == 'UNSUBSCRIBE':
            with self.server.lock:
                self.server.listeners.pop(self.client_address, None)
        elif re.fullmatch(r'Packet #[1-9][0-9]*', text):
            seq = int(text.split('#')[1])
            if self.server.drop_every and seq % self.server.drop_every == 0:
                logging.info('UDP DROP client=%s seq=%d', self.client_address, seq)
                return
            sock.sendto(encode(dict(type='ACK', seq=seq)), self.client_address)
        else:
            sock.sendto(encode(dict(type='ERROR', error='UNKNOWN_DATAGRAM')), self.client_address)


class UDPServer(socketserver.UDPServer):
    allow_reuse_address = True

    def __init__(self, address, drop_every=10, interval=2):
        super().__init__(address, UDPHandler)
        self.drop_every = drop_every
        self.interval = interval
        self.listeners = {}
        self.lock = threading.Lock()
        self.stopped = threading.Event()

    def alerts(self):
        seq = 0
        while not self.stopped.wait(self.interval):
            seq += 1
            with self.lock:
                now = time.monotonic()
                self.listeners = {a: t for a, t in self.listeners.items() if now-t < 300}
                targets = list(self.listeners)
            for target in targets:
                try:
                    self.socket.sendto(encode(dict(type='EMERGENCY_ALERT', seq=seq,
                        message='MOCK: storm warning', mock=True)), target)
                    logging.info('ALERT seq=%d target=%s', seq, target)
                except OSError:
                    logging.exception('Alert delivery error')


def serve(args):
    logging.basicConfig(level=logging.INFO, format='%(asctime)s %(threadName)s %(message)s',
                        handlers=[logging.StreamHandler(), logging.FileHandler(args.log,
encoding='utf-8')])
    with TCPServer((args.host, args.tcp_port), TCPHandler) as tcp, UDPServer(
            (args.host, args.udp_port), args.drop_every, args.interval) as udp:
        threads = [threading.Thread(target=tcp.serve_forever, daemon=True),
                   threading.Thread(target=udp.serve_forever, daemon=True),
                   threading.Thread(target=udp.alerts, daemon=True)]
        for thread in threads:
            thread.start()
        logging.info('READY TCP=%s UDP=%s drop_every=%d', tcp.server_address, udp.server_address,
args.drop_every)
        try:
            while True:
                time.sleep(.2)
        except KeyboardInterrupt:
```

```
            pass
        finally:
            udp.stopped.set()
            tcp.shutdown()
            udp.shutdown()


def tcp_client(args):
    # connect creates a TCP session; context manager always closes the socket.
    with socket.create_connection((args.host, args.tcp_port), timeout=5) as sock:
        stream = sock.makefile('rb')
        try:
            while True:
                try:
                    text = input('weather> ')
                except EOFError:
                    text = 'EXIT'
                sock.sendall((text + '\n').encode('utf-8'))
                raw = stream.readline(MAX_LINE + 1)
                if not raw:
                    print('Server closed connection')
                    break
                print(raw.decode('utf-8').strip())
                if json.loads(raw).get('status') == 'BYE':
                    break
        finally:
            stream.close()


def udp_probe(args):
    rows = []
    target = (socket.gethostbyname(args.host), args.udp_port)
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
        for seq in range(1, 101):
            start = time.monotonic()
            deadline = start + 1.0
            sock.sendto(f'Packet #{seq}'.encode(), target)
            received = False
            while time.monotonic() < deadline:
                sock.settimeout(max(.001, deadline-time.monotonic()))
                try:
                    raw, source = sock.recvfrom(8192)
                    value = json.loads(raw)
                    if source == target and value.get('type') == 'ACK' and value.get('seq') == seq:
                        received = True
                        break
                except socket.timeout:
                    break
                except (ValueError, UnicodeDecodeError):
                    continue
            elapsed = (time.monotonic()-start)*1000
            rows.append(dict(seq=seq, received=received, elapsed_ms=round(elapsed, 3)))
            print(f'Packet #{seq}: ' + (f'ACK RTT={elapsed:.3f} ms' if received else 'TIMEOUT (1
s)'))
    lost = sum(not row['received'] for row in rows)
    result = dict(sent=100, received=100-lost, lost=lost, loss_pct=lost, timeout_s=1, packets=rows)
    print(json.dumps({k: v for k, v in result.items() if k != 'packets'}))
    if args.output:
        with open(args.output, 'w', encoding='utf-8') as f:
            json.dump(result, f, indent=2)
    return result


def alert_client(args):
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
        sock.settimeout(max(5, args.interval * 3))
        target = (args.host, args.udp_port)
        sock.sendto(b'SUBSCRIBE', target)
        try:
            while True:
                raw, source = sock.recvfrom(8192)
                print(source, raw.decode().strip())
        except KeyboardInterrupt:
```

```
            pass
        finally:
            sock.sendto(b'UNSUBSCRIBE', target)


def main():
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument('mode', choices=['server', 'tcp', 'udp', 'alerts'])
    p.add_argument('--host', default='127.0.0.1')
    p.add_argument('--tcp-port', type=int, default=9000)
    p.add_argument('--udp-port', type=int, default=9001)
    p.add_argument('--drop-every', type=int, default=10)
    p.add_argument('--interval', type=float, default=2)
    p.add_argument('--log', default='server.log')
    p.add_argument('--output')
    args = p.parse_args()
    if args.drop_every < 0 or args.interval <= 0:
        p.error('drop-every >= 0 and interval > 0 required')
    try:
        {'server': serve, 'tcp': tcp_client, 'udp': udp_probe, 'alerts':
alert_client}[args.mode](args)
    except (OSError, ValueError) as exc:
        p.exit(1, f'Error: {exc}\n')


if __name__ == '__main__':
    main()
```

Додаток Б. Інтеграційні тести

test_weather.py

```
"""Real loopback integration tests, including simultaneous open sessions."""
import concurrent.futures
import json
import socket
import threading
import time
import unittest
import weather_mock as w


class Integration(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.tcp = w.TCPServer(('127.0.0.1', 0), w.TCPHandler)
        cls.udp = w.UDPServer(('127.0.0.1', 0), 10, .1)
        for target in (cls.tcp.serve_forever, cls.udp.serve_forever, cls.udp.alerts):
            threading.Thread(target=target, daemon=True).start()

    @classmethod
    def tearDownClass(cls):
        cls.udp.stopped.set()
        for server in (cls.tcp, cls.udp):
            server.shutdown()
            server.server_close()

    def test_four_parallel_clients(self):
        barrier = threading.Barrier(4)
        def client(city):
            with socket.create_connection(self.tcp.server_address) as s:
                s.settimeout(3)
                with s.makefile('rb') as f:
                    barrier.wait(timeout=3)
                    s.sendall(f'GET_WEATHER {city}\n'.encode())
                    reply = json.loads(f.readline())
                    barrier.wait(timeout=3)
                    s.sendall(b'EXIT\n')
                    self.assertEqual(json.loads(f.readline())['status'], 'BYE')
                    self.assertEqual(f.readline(), b'')
                    return reply
        with concurrent.futures.ThreadPoolExecutor(4) as pool:
            replies = list(pool.map(client, ['Kyiv', 'Lviv', 'Odesa', 'Dnipro']))
        self.assertEqual(len(replies), 4)
        self.assertTrue(all(r['status'] == 'OK' and r['mock'] for r in replies))

    def test_framing_errors_unicode(self):
        with socket.create_connection(self.tcp.server_address) as s, s.makefile('rb') as f:
            s.sendall(b'GET_WEA')
            s.sendall('THER Київ\nBAD\nGET_WEATHER
123\n\xff\nEXIT\n'.encode('utf-8').replace(b'\xc3\xbf', b'\xff'))
            values = [json.loads(f.readline()) for _ in range(5)]
            self.assertEqual(values[0]['city'], 'Київ')
            self.assertEqual([r.get('error') for r in values[1:4]],
                             ['UNKNOWN_COMMAND', 'INVALID_CITY', 'INVALID_UTF8'])
            self.assertEqual(values[-1]['status'], 'BYE')

    def test_long_frame(self):
        with socket.create_connection(self.tcp.server_address) as s, s.makefile('rb') as f:
            s.sendall(b'x' * (w.MAX_LINE + 1) + b'\n')
            self.assertEqual(json.loads(f.readline())['error'], 'FRAME_TOO_LONG_OR_INCOMPLETE')
            self.assertEqual(f.readline(), b'')

    def test_udp_drop_and_recovery(self):
        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
            s.settimeout(1)
            s.sendto(b'Packet #10', self.udp.server_address)
            start = time.monotonic()
            with self.assertRaises(socket.timeout):
```

```
                s.recvfrom(8192)
            self.assertGreaterEqual(time.monotonic()-start, .95)
            s.sendto(b'Packet #11', self.udp.server_address)
            self.assertEqual(json.loads(s.recvfrom(8192)[0])['seq'], 11)

    def test_alert_fanout(self):
        sockets = [socket.socket(socket.AF_INET, socket.SOCK_DGRAM) for _ in range(2)]
        try:
            for s in sockets:
                s.settimeout(2)
                s.sendto(b'SUBSCRIBE', self.udp.server_address)
                self.assertEqual(json.loads(s.recvfrom(8192)[0])['type'], 'SUBSCRIBED')
            for s in sockets:
                self.assertEqual(json.loads(s.recvfrom(8192)[0])['type'], 'EMERGENCY_ALERT')
                s.sendto(b'UNSUBSCRIBE', self.udp.server_address)
        finally:
            for s in sockets:
                s.close()


if __name__ == '__main__':
    unittest.main(verbosity=2)
```